Skill Trees for AI Agents: Hierarchical Tool Selection via Self-Improving Graphs

Akshay Singh
akshay@obelysk.org
Obelysk

Bhargavi G.
bhargavi@obelysk.org
Obelysk

Anant Singh
anant@obelysk.org
Obelysk

Abstract

As LLM agents interface with increasingly large tool inventories, tool selection accuracy degrades. Existing approaches present a stark trade-off: flat embedding retrieval is computationally efficient but suffers from low precision, whereas LLM-based classification provides high accuracy but introduces high latency and token overhead per query. To address this, we introduce **Skill Trees**, a hierarchical routing architecture that organizes tools into a directed acyclic graph (DAG) of skill domains. Rather than relying on long system descriptions, domains and tools are indexed via short exemplar queries, enabling a two-level traversal that scales logarithmically. A Temporal Exponential Moving Average (EMA) learning mechanism continuously updates graph edges based on execution traces, reinforcing co-used tool patterns while decaying stale edges. Implemented within a local desktop agent environment, Skill Trees narrow a candidate pool of 200+ tools to an adaptive subset in under 2 ms. On a 300-query benchmark, this approach improves F1 by 15.7% over flat retrieval and achieves recall parity with state-of-the-art LLMs (e.g., Claude Opus 4) while reducing routing latency by three orders of magnitude (1,025×). These results suggest that structured, adaptive retrieval can serve as a viable pre-execution routing layer, mitigating both the tool scaling bottleneck and the overhead of LLM-based classification.

1 Introduction

As Large Language Models (LLMs) transition from passive conversational assistants to active participants in user workflows, interaction with external environments has become a focal point of agent architecture. Agents bridge the gap between text generation and concrete action through tool use, allowing them to read files, search the web, and operate application UIs. The standard architecture provides the LLM with a flat list of available tools alongside their text descriptions, relying on the model's in-context reasoning to select the appropriate tool for a given query.

1.1 The Tool Scaling Bottleneck

While this flat-list approach is sufficient for narrow, domain-specific applications requiring only a handful of tools, it encounters a severe scaling limit in general-purpose agents. As capabilities expand to encompass diverse workflows—from software development to daily productivity—the tool inventory necessarily grows. This growth triggers a measurable accuracy degradation often termed the *tool explosion problem* (Table 1).

This degradation is not primarily a model limitation, but rather reflects the underlying complexity of selecting from a large, flat inventory where semantic descriptions overlap and the model must evaluate every option within its context window. The problem is exacerbated by user-extensible tool architectures (e.g., the Model Context Protocol [13]), where the set of available tools is dynamic and potentially unbounded.

Table 1: Tool selection accuracy versus inventory size.

Available Tools	Selection Accuracy	Source
5 tools	∼95%	AutoTool [1], ToolLLM [2]
15 tools	∼82%	Empirical (ReAct agents)
30 tools	∼68%	AutoTool ablations
50+ tools	Sharp degradation	TURA [9]

Table 2 summarizes existing approaches and their principal limitations.

Table 2: Existing approaches to tool selection.

Approach	Method	Weakness
Full enumeration	Send all tool descriptions to LLM	Token cost scales linearly; accuracy degrades
Manual routing	Hardcoded "if code → code tools" rules	Brittle; doesn't scale to user-installed skills
Flat embedding retrieval	Embed query → cosine sim vs. all tools	No hierarchy; misses tool relationships
AutoTool [1]	Directed graph of tool transitions from logs	Flat graph; requires historical data; no pre-computation
Agent-as-a-Graph [6]	Joint tool/agent retrieval in shared vector space	No hierarchical traversal; no self-improving edges
AgentRouter [7]	GNN routing over knowledge graph	Requires training a GNN; not applicable to dynamic inventories

1.2 Hierarchical Structure and Predictive Pre-Computation

Rather than deliberating between disparate tools (e.g., `editFile` vs. `webSearch`) as equally likely candidates, execution traces in specialized workflows typically cluster by domain. Resolving the domain intent prior to tool selection provides an efficient mechanism to prune irrelevant branches early.

We formalize this intuition as a **Skill Tree**—a directed acyclic graph (DAG) in which:

- A **root** connects to **primitive domain** nodes (OS, Web, Computer Use) and **intent domain** nodes (Code, Research, Business, Creative, Personal).
- **Domain nodes** connect to tool leaf nodes, such as `editFile`, `searchFiles`, and `runCommand`.
- Tools may have **multiple parents** to reflect cross-domain utility (e.g., `editFile` belongs to both Code and OS).
- Edges carry **learned weights** that update dynamically based on usage co-occurrence.
- Each domain node stores **multiple exemplar embeddings** to facilitate maximum-similarity scoring against short user queries.

A key architectural property of Skill Trees is the temporal separation of concerns across a **three-phase pipeline**:

Phase A: Install-Time Indexing (~50 ms per skill). Tool definitions and exemplar queries are parsed, embedded, and mapped into the DAG.

Phase B: Type-Time Traversal (~15 ms, during input debounce). User input is embedded to perform a hierarchical traversal, resolving and caching the routing path before execution begins.

Phase C: Send-Time Injection (~0 ms). The cached tool subset and domain context are injected directly into the LLM payload.

1.3 Contributions

In this work, we make the following contributions:

(1) **Hierarchical Skill Graph Data Structure**: A DAG organization for AI agent tools featuring a 3-tier pack architecture (primitive, intent, fallback) that captures cross-domain dependencies while avoiding the limitations of strict taxonomic trees.
(2) **Two-Level Exemplar Embeddings**: We demonstrate that embedding short exemplar phrases rather than long descriptions at both the domain and tool levels bridges the semantic gap between user queries and API definitions, systematically improving retrieval precision.
(3) **Hierarchy Amplification of Embedding Quality**: An empirical analysis revealing that hierarchical traversal amplifies embedding model improvements at a 45× greater magnitude than flat retrieval, while exposing an asymmetric interaction between embedding dimensionality and exemplar coverage limits.
(4) **Temporal EMA Edge Learning**: A weight update formulation combining differential reinforcement with exponential temporal decay, enabling the routing graph to continuously adapt to shifting user workflows without hard caps or manual scheduling.
(5) **Three-Phase Routing Pipeline**: A systems architecture separating index, traversal, and injection, which eliminates LLM classification dependencies for > 80% of requests and reduces routing latency from ~200–500 ms to ~2 ms.
(6) **Open-source implementation and deployment** as the sole routing mechanism in a personal AI agent with 18 tools across 8 domains.

2 Related Work

2.1 Tool Selection for LLM Agents

Full enumeration remains the dominant approach: every available tool description is included in the LLM prompt. This works at small scale (5–10 tools) but degrades at 30+ tools due to attention dilution, token cost, and tool-calling hallucinations [2] (ToolLLM).

AutoTool [1] introduces a directed graph built from historical tool usage traces, leveraging "tool usage inertia"—the observation that tool invocations follow predictable sequential patterns. AutoTool reduces LLM inference cost by ~30% by predicting the next tool from graph transitions. However, AutoTool's graph is *flat* (no hierarchy), requires substantial historical data to build, and does not perform pre-computation during user input.

Table 3 provides a structured comparison between AutoTool and Skill Trees, as both use graph structures for tool selection but differ fundamentally in their approach.

Table 3: Structural comparison: AutoTool vs. Skill Trees.

Dimension	AutoTool	Skill Trees (ours)
Graph structure	Flat directed graph (tool → tool transitions)	Hierarchical DAG (root → domain → tool)
Data requirements	Historical agent trajectories	Zero (exemplars at install-time)
Selection method	Sequential: given current tool, predict next	Parallel: given query, select entire tool subset
Pre-computation	None	Type-time traversal during keystroke debounce
LLM fallback	Yes	No—embedding-only
Cost reduction	~30% fewer LLM calls	100% routing inference eliminated
Adaptation	Static after construction	Continuous: Temporal EMA

The key architectural distinction is *sequential vs. parallel*: AutoTool predicts the *next* tool based on the *current* tool, while Skill Trees select an *entire tool subset* from a single query. The two approaches are complementary—Skill Trees could provide the initial tool set, and AutoTool could refine subsequent selections during multi-step execution.

Recent frameworks extend the graph-based paradigm to execution resilience. **Graph-Based Self-Healing Tool Routing** [20] treats agent control-flow entirely as a routing problem, utilizing Dijkstra's algorithm over cost-weighted tool graphs to recover from tool failures deterministically without invoking the LLM. While this addresses execution-stage fault tolerance, our Skill Trees target the preceding step: hierarchical pre-execution selection to define the initial graph namespace.

TURA [9] benchmarks tool-use accuracy across varying tool counts and identifies systematic degradation patterns. Our work builds on TURA's empirical findings to motivate hierarchical traversal as a structural solution.

2.2 Graph-Based Tool and Agent Retrieval

Graph-based routing architectures emerged recently as the primary solution to multi-model and multi-agent coordination. **Agent-as-a-Graph** [6] maps tools and parent agents into a unified knowledge graph for joint retrieval. However, this approach relies on flat traversal without hierarchical narrowing and lacks a continuous pre-computation pipeline.

To manage complexity, several systems now employ heterogeneous graph networks. **AgentRouter** [7] and **GraphRouter** [21] formulate query coordination as an inductive knowledge graph problem, utilizing Graph Neural Networks (GNNs) to predict query-agent or query-LLM compatibility based on past interactions. While highly effective for optimizing cost and performance across different backbone models, these systems require trained GNNs—a heavy dependency poorly suited for dynamic, local desktop environments with user-extensible tool inventories.

Hierarchical constraints are also gaining traction. **HAPS** (Hierarchical LLM Routing) [22] employs a high-level router to select LLM architectures and a low-level router for parameter allocation, demonstrating that hierarchical query-complexity routing outperforms flat baselines. Similarly, **SkillRL** [12] introduces a hierarchical skill library (SkillBank) that distills successful agent trajectories into reusable behavioral patterns via reinforcement learning. **A-RAG** [8] provides LLMs with hierarchical retrieval interfaces for adaptive search, but focuses on document retrieval, not tool selection.

While SkillRL, HAPS, and Skill Trees all exploit hierarchy, they address orthogonal bottlenecks: SkillRL optimizes *execution strategy selection* through RL training, HAPS optimizes *model architecture selection*, and Skill Trees optimize *zero-training tool routing* at install time. These approaches are highly complementary; a system could utilize Skill Trees to provide the initial bounded toolset, and SkillRL to guide the agent's strategy within that subset.

2.3 Intent Classification for Agents

Traditional approaches use a dedicated LLM call for intent classification before tool selection [17] (LangChain), adding 200–500 ms latency per request and consuming tokens. Some systems use structured output with schema validation for classification, which improves reliability over free-text parsing but still requires an LLM call.

Our approach folds domain detection, tool selection, execution mode determination, and step budget allocation into a single embedding-based DAG traversal, eliminating the classification call entirely.

2.4 Embedding-Based Retrieval for Tools

Flat embedding retrieval—embed the query, compute cosine similarity against all tool description embeddings, take top-K—is a natural baseline. It is fast (~5 ms for small inventories) and requires no training. However, it treats all tools as equally likely candidates (no domain pruning), misses tool relationships, does not improve over time (static embeddings), and accuracy at 50+ tools approaches random for semantically similar tools. Our hierarchical approach addresses these four limitations.

2.5 Tool Selection Evaluation

ToolLLM [2] provides a large-scale benchmark (16,000+ APIs, commonly referred to as ToolBench) for evaluating tool-augmented LLMs. **MetaTool** [3] evaluates tool awareness with 21,000+ queries testing single- and multi-tool selection. **BFCL** [5] focuses on function-calling accuracy, and **ToolSandbox** [4] evaluates stateful, conversational tool use.

Our benchmark is smaller in scale (300 queries, 18 tools) but specifically designed for the *routing-layer* evaluation problem: given a user query before any tools have been invoked, which tools should be made available? This differs from ToolBench/MetaTool's focus on *execution-time* tool invocation within multi-step trajectories, and from ToolRet's [10] focus on retrieval model evaluation over 43,000 tools. We include adversarial zero-tool queries and cross-domain tasks that existing benchmarks do not specifically target.

3 Skill Tree Architecture

3.1 Data Structures

The Skill Tree is built from three core data structures.

SkillNode. Each node in the DAG represents either a domain or a tool:

Listing 1: SkillNode interface (TypeScript).

```
interface SkillNode {
  id: string;            // "domain:code", "tool:editFile
     "
  name: string;          // "Code", "Edit File"
  type: 'root'|'domain'|'subdomain'|'tool';
  description: string;   // semantic description
  embedding: Float32Array; // 384-dim (MiniLM-L6-v2)
  exemplarEmbeddings?: number[][]; // multiple exemplars
  parents: string[];     // DAG: multiple parents
  children: string[];    // sub-domains or tools
  enabled: boolean;      // user can toggle on/off
  usageCount: number;    // lifetime usage
  lastUsed: number;      // timestamp
  packId?: string;       // source skill pack
  toolName?: string;     // leaf nodes: actual tool name
}
```

SkillEdge. Each edge carries a learned weight and type:

Listing 2: SkillEdge interface (TypeScript).

```
interface SkillEdge {
  source: string;        // source node ID
  target: string;        // target node ID
  weight: number;        // 0-1, learned
  type: 'hierarchy'|'semantic'|'cooccurrence'|'pipeline';
  reinforcements: number;// times reinforced
  lastReinforced: number;// timestamp
}
```

Four edge types capture different relationships (Table 4).

SkillPath. The output of traversal:

Listing 3: SkillPath interface (TypeScript).

Table 4: Edge types in the Skill Tree.

Type	Created By	Evolved By
`hierarchy`	Install-time: parent → child in DAG	Never decayed (structural)
`semantic`	Install-time: cosine sim > 0.6	Decayed if unused
`cooccurrence`	Usage: tools used together	Reinforced on co-use, decayed over time
`pipeline`	Usage: tool B follows tool A	Reinforced on sequential use

```
interface SkillPath {
  tools: string[];       // focused tool names
  primaryDomain: string; // e.g. "code"
  secondaryDomains: string[];
  executionMode:
    'stream'|'generate'|'background';
  stepBudget: number;
  confidence: number;
  systemPromptFragments: string[];
  isRecurring: boolean;
  goal: string;
}
```

3.2 Graph Construction (Install-Time Indexing)

When a **Skill Pack** is installed (built-in or from marketplace):

(1) **Parse**: Read `SKILL.md` (YAML frontmatter + markdown body) → extract name, description, tool list, system prompt, evaluation rubric.
(2) **Embed exemplars**: For each domain, embed 5–13 short **exemplar queries** (not the pack description) using `all-MiniLM-L6-v2` (Sentence-BERT [15], local, ~5 ms per embedding, pipeline-based via `@huggingface/transformers`). Store all embeddings as `exemplarEmbeddings` on the domain node. The primary `embedding` field is the L2-normalized average, used for backward compatibility.
Why exemplars, not descriptions: MiniLM-L6-v2 is a sentence-pair model [15]—it produces near-zero cosine similarity (measured: 0.04) between short user queries and long domain descriptions. Short-vs-short exemplar comparison yields 10–12× higher similarity scores, fixing ~30% of domain misclassification.
(3) **Embed tools**: Generate embeddings for each tool description: `"${toolName}: ${toolDescription}"`.
(4) **Build hierarchy**: Create root → domain → tool hierarchy edges. Tools shared across packs get multiple parents (DAG structure).
(5) **Build semantic edges**: Create `semantic` edges between tools with embedding similarity > 0.6.
(6) **Persist**: Save the updated graph to `~/.niom/skill-graph.json` (versioned format with dirty-tracking for efficient writes).

3-Tier Pack Architecture. The built-in packs follow a layered design (Table 5).

Primitive packs own tool *implementations* (`readFile`, `webSearch`, `mouseClick`). Intent packs own domain-specific *high-level tools* (`deepResearch`, `searchFiles`) and provide the system prompt personality. The OS primitive is always included in traversal (§3.3), ensuring every query has access to file operations.

Table 5: 3-Tier pack architecture.

Tier	Packs	Tools	Role
Primitive	OS, Web, Computer Use	17	System-level
Intent	Code, Research, Business, Creative, Personal	5 unique	Domain reasoning
Fallback	General	All	Ambiguous queries

3.3 Hierarchical Traversal (Type-Time Pre-Computation)

The traversal algorithm runs on a debounced keystroke timer (300 ms) and resolves the full SkillPath in ~1.5 ms (measured).

Algorithm 1 Hierarchical Skill Tree Traversal (Type-Time)

Require: query q (partial or complete)
Ensure: SkillPath (tools, domain, mode)
1: $\mathbf{e}_q \leftarrow \textsc{Embed}(q)$
2: **if** q matches greeting pattern **then**
3: **return** General pack
4: **end if**
5: **if** LRU cache hit (50 entries, 30s TTL) **then**
6: **return** cached result
7: **end if**

Domain Scoring (max-sim):
8: **for** each domain node $d \in \mathcal{D}$ **do**
9: **if** d.exemplarEmbeddings exist **then**
10: $s_d \leftarrow \max_{e \in d.\text{ex}} \cos(\mathbf{e}_q, e)$
11: **else**
12: $s_d \leftarrow \cos(\mathbf{e}_q, \mathbf{e}_d)$
13: **end if**
14: $s_d \leftarrow s_d + \textsc{UsageBoost}(d)$
15: **end for**
16: $\mathcal{D}_{\text{top}} \leftarrow \text{top-3}(s_d)$
17: **if** os $\notin \mathcal{D}_{\text{top}}$ **then**
18: $\mathcal{D}_{\text{top}} \leftarrow \mathcal{D}_{\text{top}} \cup \{\text{os}\}$
19: **end if**

Tool Scoring (drill into top domains):
20: **for** each tool $t \in \mathcal{D}_{\text{top}}$.children **do**
21: $\text{sim} \leftarrow \cos(\mathbf{e}_q, \mathbf{e}_t)$
22: $\text{boost} \leftarrow s_{d(t)} \times 0.1 + \mathbb{1}[t \in d_{\text{pri}}] \times 0.05$
23: $s_t \leftarrow \text{sim} + \text{boost} + w_{\text{coocc}}(t_{\text{last}}, t)$
24: **end for**

Adaptive Cutoff:
25: $\tau \leftarrow \mu(s_t) + 0.25 \cdot \sigma(s_t)$
26: $\mathcal{T} \leftarrow \{t : s_t \geq \tau\}$, clamped to $[3, 6]$
27: Build SkillPath, cache (LRU, 30s TTL)
28: Run post-routing validation (Tier 1/2, §3.5)
29: **return** SkillPath

Embedding: ~5 ms local ONNX. UsageBoost: logarithmic, ≤ 0.1. OS domain always guaranteed in $\mathcal{D}_{\text{top}}$.

Two-Level Exemplar Embeddings (Step 4 in Algorithm 1). The same short-to-short embedding insight used for domain matching is applied recursively to tools. Instead of embedding `writeFile` with its technical API description ("Create or overwrite a file with the given content…"), we embed 2–4 short usage-oriented phrases ("save this report to a file", "create a new document with this content"). At query time, tool scoring uses max-sim across these exemplars. This single change recovered `writeFile` selection from

3/20 to 17/20 in business queries—purely through better embedding inputs, with no tool names hardcoded.

Tiered Adaptive Cutoff. The 3-tier pack architecture naturally defines two cutoff levels. Tools from primitive domains (OS, Web, Computer Use—defined in `PRIMITIVE_DOMAINS`) use a relaxed cutoff (μ) while intent-domain tools use a strict cutoff ($\mu + 0.25\sigma$). This ensures foundational tools pass more easily without flooding the selection. No specific tools are hardcoded—any tool added to a primitive pack automatically gets relaxed treatment.

Domain-Dependent Primitive Guarantees.

- OS domain: always included (file operations are universal).
- Web domain: injected when primary domain is research, business, or creative (these domains almost always need `webSearch`/`fetchUrl`).

Comparison count. Measured on the 300-query benchmark (Table 6).

Table 6: Embedding comparisons per query.

Component	Count
Domain scoring (8 domains × 5–13 exemplars)	~56
Tool scoring (top 3–5 domains × 3–9 tools × 2–4 exemplars)	~42
Total	**~98**
Flat baseline (all tools)	~19

While the two-level exemplar scoring increases comparison count (~98 vs. ~19 for flat), the increase is bounded by the number of exemplars rather than the number of tools, and the absolute cost remains well under 2 ms. The accuracy improvement (+15.7% F1 on the 300-query benchmark) represents a favorable tradeoff.

3.4 Edge Weight Learning

After each successful tool execution, the system records the ordered tool sequence and updates edge weights using **Temporal EMA**, a learning rule that combines two complementary properties.

1. Differential reinforcement. (diminishing returns): each reinforcement adds proportionally less as the weight grows, preventing saturation:

$$\Delta w = \alpha \cdot (1 - w_t) \tag{1}$$

When $w_t = 0.3$, $\Delta w \approx 0.105$. When $w_t = 0.9$, $\Delta w \approx 0.015$. The weight asymptotically approaches 1.0 but never reaches it—no hard cap.

2. Temporal de-learning. (exponential decay): before each reinforcement, the current weight is decayed based on time elapsed since the last use:

$$w'_t = w_t \cdot e^{-\lambda \cdot \Delta\tau} \tag{2}$$

where $\lambda = \ln(2)/\tau_{\text{half}}$. With a 30-day half-life, an edge unreinforced for 30 days loses half its weight. An edge unreinforced for 60 days retains only 25%.

Combined update rule. For each tool pair (co-occurrence) and sequential pair (pipeline) in the recorded sequence, let $\Delta\tau$ be the time elapsed since the last reinforcement. The new weight w_{t+1} is computed as:

$$w_{t+1} = w_t \cdot e^{-\lambda \cdot \Delta\tau} + \alpha \cdot \left(1 - w_t \cdot e^{-\lambda \cdot \Delta\tau}\right) \tag{3}$$

Why not naïve linear? The standard approach $w \leftarrow \min(1.0,\ w + 0.1)$ saturates in ~7 reinforcements, creating two problems: (1) the system stops learning after initial convergence, and (2) stale patterns remain at maximum weight indefinitely. In our benchmark, linear weights hit 1.0 by Day 3 of simulated usage and cannot adapt to changing user behavior. Temporal EMA maintains a continuously adaptive signal.

The co-occurrence bonus influences traversal scoring in Phase B (Steps 4–5 of Algorithm 1), creating a feedback loop: frequently co-used tools bubble up in future traversals, improving co-selection accuracy over time. Crucially, tools that *stop* being co-used naturally de-weight, allowing the system to track evolving user workflows.

3.5 Two-Tier Post-Routing Validation

Embedding-based traversal correctly handles ~90% of requests. The remaining ~10% involve *intent modifiers*—secondary signals that change HOW the result should be delivered but do not change WHAT domain/tools are needed.

Example. "Send AI-generated social media posts *every day*"—the Skill Tree correctly identifies Creative/Business tools but misses the scheduling intent because "every day" is semantically weak compared to the business/creative domain signals.

Tier 1: Schedule Signal Detector. (~0 ms). A regex pattern matches temporal language:

```
/\b(every\s*(day|week|morning|evening|hour)
  |daily|weekly|remind\s*me
  |keep\s*monitoring|recurring)\b/i
```

Runs on every request independently of traversal scores. Overrides `executionMode` to `background` when matched.

Tier 2: Intent Validator. (~300 ms, conditional).

- *Fires when*: top-2 domain scores within 0.1, weak temporal language, multi-clause queries.
- *Does NOT fire when*: high-confidence single domain (> 0.75), Tier 1 already handled, simple queries (< 40 chars).
- Uses cheapest available LLM (`extraction` model role) with structured output.
- Only applies override if confidence > 0.7.

Fire rate. Tier 2 fires on $< 20\%$ of requests in production usage. The vast majority (80%+) route through the fast path with zero LLM calls before execution.

4 Implementation

4.1 System Context

Skill Trees are implemented as the sole routing mechanism in NIOM, an OS-level personal AI agent built as:

- **Tauri/Rust shell** (~200 lines of code): system tray, global hotkeys, overlay window management.
- **Node.js sidecar**: AI SDK v6 [16], agent engine, tools, Skill Tree, task management, MCP client [13].
- **React frontend**: HUD-style overlay UI with real-time SSE streaming.

The entire system runs locally. All data is stored at `~/.niom/` with AES-256-GCM encryption.

4.2 Embedding Model

Table 7 summarizes the production embedding configuration.

Table 7: Embedding model configuration.

Property	Value
Model	`all-MiniLM-L6-v2` [15]
Dimensions	384
Embedding latency	~5 ms (pipeline, batched)
Cache	LRU, 500 entries
Warm-up	Lazy loading; hash-based fallback if ONNX fails
Runtime	Node.js (ONNX Runtime), no GPU required

4.3 Tool Inventory

NIOM ships with 18 built-in tools organized into the 3-tier pack architecture.

Primitive Packs. (system capabilities):

Table 8: Primitive pack tool inventory.

Pack	Domain	Tools
OS	File system & shell	`readFile`, `writeFile`, `editFile`, `listDirectory`, `searchFiles`, `deleteFile`, `runCommand`, `systemInfo` (8)
Web	Internet	`webSearch`, `fetchUrl` (2)
Computer Use	GUI automation	`screenshot`, `mouseClick`, `mouseMove`, `typeText`, `pressKey`, `scroll`, `getActiveWindow` (7)

Intent Packs. (domain reasoning):

Table 9: Intent pack tool inventory.

Pack	Domain	Unique Tools
Code	Software development	`searchFiles`, `editFile`, `readFileRange` (3)
Research	Information synthesis	`deepResearch` (1)
Business	Analysis & operations	`deepResearch` (1)
Creative	Content & media	`deepResearch` (1)
Personal	Productivity	`notifyUser` (1)
General	Fallback	All tools

The separation ensures primitive tools (`readFile`, `webSearch`) are owned by exactly one pack, while intent packs focus on high-level domain-specific tools. The traversal composes tools from multiple tiers: e.g., a business query gets `deepResearch` (Business) + `readFile`, `writeFile` (OS) + `webSearch` (Web).

Additionally, the MCP client can dynamically register tools from 13,000+ MCP servers [13], which are automatically indexed into the Skill Tree on connection.

4.4 Key Implementation Details

Graph persistence. The skill graph is saved as `~/.niom/skill-graph.json` with a version field, dirty-tracking (only write when edges change), and atomic writes (write to `.tmp` → rename) to prevent corruption.

Concurrency. The `SkillPathResolver` is a singleton with a thread-safe LRU cache. Multiple simultaneous traversals (e.g., debounced typing + agent execution) are safe—the cache is read-heavy with infrequent writes.

Graceful degradation. If the embedding model fails to load (first-time download, corrupted cache), the system falls back to hash-based pseudo-embeddings. If the Skill Tree is not initialized, the router falls back to pack-defined tool sets. All failures are caught and logged, and never crash the system.

4.5 Lines of Code

Table 10 reports the implementation size.

Table 10: Skill Tree implementation size.

Component	Lines of Code	File
SkillGraph (DAG)	380	`skills/graph.ts`
SkillPathResolver	726	`skills/traversal.ts`
Embeddings	210	`skills/embeddings.ts`
Router (pack resolver)	180	`skills/router.ts`
Intent Validator (Tier 2)	150	`skills/intent-validator.ts`
Skill Pack Loader	120	`skills/loader.ts`
Total	~1,766	—

5 Evaluation

5.1 Experimental Setup

We evaluate Skill Trees on three axes: **accuracy** (correct tool selection), **latency** (routing time), and **adaptation** (co-selection improvement over time).

5.1.1 Research Progression. The evaluation evolved through four phases, each prompted by findings from the previous one. We present results in this order rather than collapsing into a single "final numbers" table, as the progression itself reveals properties of the architecture that a polished summary would obscure (Table 11).

All code changes are traceable through the project's version-controlled repositories. §5.2 and §5.6 report the Phase 3 configuration (MiniLM, 13 exemplars) as the canonical baseline. §5.8 reports on the Phase 4 120-query development set (5 exemplars); these

Table 11: Research progression across four evaluation phases.

Phase	What we did	Key finding	Ref
1: Architecture	11 runs on 120-query dev set	Two-level exemplar embeddings dominant (+29.9% F1)	§5.2, §5.5
2: Scale-up	300 queries; LLM & flat baselines	Recall parity with LLM (0.646 vs. 0.650); 1,025× faster	§5.2, §5.6
3: Ablation	2×2 ablation (model × exemplar)	Hierarchy amplifies quality (45×); asymmetric interaction	§5.7
4: Precision	Four precision filters on 120-query dev set	Similarity banding → fine-tuning needed	§5.8

precision improvements have not yet been re-evaluated on the 300-query benchmark.

Benchmark. The evaluation consists of a 120-query development set used for architectural routing evaluation (Phases 1-2, 4), and a subsequent 300-query expanded evaluation set used to validate scalability against adversarial and cross-domain edge cases (Phase 3). The 300-query benchmark spans 6 domains (50 per domain), manually labeled with ground-truth tool sets (0–6 tools per query). Queries range from adversarial zero-tool inputs ("Hello!", "Yes", "Give me some motivation") to complex cross-domain multi-step tasks ("Research the topic, write a report, find relevant code examples, and update the documentation"). This expanded benchmark explicitly probes edge cases: negation ("don't modify anything"), one-word responses ("Continue"), emotional queries, and ambiguous cross-domain requests.

Baselines.

(1) **Full enumeration**: Send all 15+ tool descriptions to the LLM (GPT-4o, Claude 3.5 Sonnet).
(2) **Flat embedding retrieval**: Embed query → top-K cosine similarity against all tool embeddings.
(3) **LLM classification**: Dedicated `generateObject()` call to classify domain + tools (GPT-4o-mini).
(4) **AutoTool-style**: Directed graph of tool transitions (built from the same usage data).

Metrics.

- **Precision@K**: fraction of selected tools in the ground truth.
- **Recall@K**: fraction of ground-truth tools that were selected.
- **F1@K**: harmonic mean of precision and recall, K = selected tool count.
- **Routing latency**: time from query to tool set resolution (ms).
- **LLM calls**: number of LLM API calls consumed for routing (not execution).

5.2 Results: Skill Tree vs. Flat Baseline

We ran 11 iterative benchmark runs on the initial 120-query dataset, then validated on an expanded 300-query benchmark (Tables 12 and 13).

Table 12: 120-query development set (original).

Method	Precision	Recall	F1	Latency	LLM
Flat embedding	0.243	0.754	0.345	0.03 ms	0
Skill Tree	**0.331**	**0.821**	**0.448**	**1.67 ms**	**0**
Improvement	+36.2%	+8.9%	+29.9%	+1.67 ms	0

Table 13: 300-query evaluation set (expanded, includes adversarial).

Method	Precision	Recall	F1	Latency	LLM
Flat embedding	0.240	0.714	0.332	0.03 ms	0
Skill Tree	**0.304**	**0.660**	**0.384**	**1.82 ms**	**0**
Improvement	+26.7%	−7.6%	+15.7%	+1.82 ms	0

Per-domain F1 scores. (300-query benchmark, Table 14):

Table 14: Per-domain F1 scores (300-query benchmark).

Domain	F1	Queries	Notes
Personal	**0.476**	50	Notification + OS tools compose well
Code	**0.456**	50	Expanded exemplars cover DevOps/debug
Research	**0.422**	50	Web guarantee + tool exemplars
Creative	0.323	50	Cross-domain overlap reduces precision
General	0.314	50	23 zero-tool queries, avg 3.3 selected
Business	0.316	50	Research ↔ business overlap

Several observations emerge from the expanded evaluation:

- On the original 120-query set, Skill Trees outperform flat retrieval by +29.9% F1. On the harder 300-query benchmark, the improvement narrows to +15.7%—a more conservative but more honest measure of generalization. The additional ∼2 ms of routing latency over flat retrieval remains imperceptible in practice, as both approaches complete well within a single keystroke interval.
- **Adversarial queries are the primary weakness.** Of 23 zero-tool queries (greetings, one-word responses, meta questions), the system selects an average of 3.3 tools—a false-positive rate that directly reduces General domain F1. This highlights the need for a confidence-based tool rejection threshold.
- Two-level exemplar embeddings (domains and tools) remain the primary architectural insight. The improvement holds across both benchmark scales.

- **Expanded exemplars**: Code domain exemplars were expanded from 5 to 13 (adding DevOps/infra, debugging/profiling, comprehension, and project management patterns), improving MiniLM code F1 from 0.417 to 0.456 (+9.4%). The interaction between exemplar coverage and model quality is analyzed in §5.7.
- No tool names are hardcoded in the selection logic; the tiered cutoff from `PRIMITIVE_DOMAINS` combined with tool exemplars produces this improvement through better embedding inputs alone.

Note on F1 interpretation: The absolute F1 of 0.384 reflects a precision-recall trade-off appropriate for a routing pre-filter—a component whose role is to ensure the right tools are *available* to the execution-stage LLM, not to make the final selection. On recall—the metric that captures tool *availability*—Skill Trees achieve parity with an LLM baseline (0.660 vs. 0.650; see §5.6 and §6.4 for the full comparison and asymmetric cost analysis).

5.3 Results: Adaptation Over Time

We simulate 30 days of usage (200 queries/day across 20 weighted usage patterns from 6 domains, 6,000 total interactions) and measure co-selection accuracy—the fraction of expected tools in the selected set—on a held-out set of 17 representative queries at 7 checkpoints. Temporal EMA uses virtual time with $\alpha = 0.15$ and 30-day half-life (Table 15).

Table 15: Co-selection accuracy over 30 simulated days.

Day	Queries	Co-Selection Accuracy	Δ	Avg Weight
0	0	0.765	—	0
1	200	0.853	+11.5%	0.762
3	600	0.853	+11.5%	0.937
7	1,400	0.873	+14.1%	0.976
14	2,800	0.873	+14.1%	0.982
21	4,200	0.873	+14.1%	0.978
30	6,000	0.873	+14.1%	0.980

Several patterns emerge:

(1) **Gradual learning curve**: Co-selection accuracy improves 82% of the way (+11.5%) within the first day's 200 queries, reaching the full +14.1% improvement by Day 7. Edge weights grow from 0.762 (Day 1) to 0.976 (Day 7), exhibiting the expected diminishing returns from differential reinforcement. Per-domain gains vary: Code +47.4% (0.633 → 0.933), General +20% (0.833 → 1.000).

(2) **Weight equilibrium**: After Day 14, average weights stabilize around 0.98—not at 1.0, because the daily temporal decay ($e^{-\lambda \cdot 1} \approx 0.977\times$ per day) balances against daily reinforcement. This steady-state equilibrium means the system remains responsive to change.

(3) **De-learning capability**: An edge unreinforced for 30 days decays to half its weight ($\lambda = \ln 2/30 \approx 0.023$/day). This allows the system to track evolving workflows—a developer switching from Python to Rust will see `editFile` → `runCommand(pytest)` decay while `editFile` → `runCommand (cargo test)` strengthens. Linear weights have no such mechanism.

(4) **Differential growth curve**: The EMA increment follows $\alpha \cdot (1-w)$ with $\alpha = 0.15$: first reinforcement 0 → 0.15 (large), 10th ∼0.80 → ∼0.83 (moderate), 100th ∼0.98 → ∼0.983 (tiny). This mirrors the information-theoretic intuition that early observations carry more signal than later confirmations.

5.3.1 De-learning: Adapting to Behavior Change. To validate the temporal decay mechanism, we run a **phase-shift simulation**: 15 days of heavy code patterns (`readFile` → `editFile` → `runCommand`) followed by a pivot to 15 days of research/writing (`deepResearch` → `webSearch` → `writeFile`). We track 4 code-era edges and 4 research-era edges across the transition (Table 16).

Table 16: Phase-shift simulation: code → research pivot.

Day	Phase	Code Edges	Research Edges
0	—	0.000	0.000
1	Code	0.997	0.000
15	Code	**0.997** (peak)	0.000
16	Research	0.975 ↓	0.992 ↑
18	Research	0.931 ↓	0.998
21	Research	0.868 ↓	0.997
25	Research	0.792 ↓	0.996
30	Research	**0.705** (−29.3%)	**0.996**

Key findings:

(1) **Validated half-life**: Code edges decay from 0.997 to 0.705 over 15 unreinforced days—a 29.3% reduction, matching the theoretical prediction ($e^{-0.023\times 15} \approx 0.707$).

(2) **Asymmetric transition**: New research edges reach 0.992 within a single day (200 queries), while old code edges take 15 days to decay to 0.705—a deliberate design choice. Learning is fast; de-learning is gradual.

(3) **Contrast with naïve linear**: Under $\min(1.0, w+0.1)$, code edges would remain at **1.0 forever** after the pivot—the system would never adapt. Temporal EMA provides the only path to de-learning stale patterns.

(4) **Practical impact**: `readFile` ↔ `editFile` starts at 0.998 (1,426 reinforcements) and decays to 0.706 by Day 30. In traversal scoring, this reduces the co-occurrence bonus from $0.998 \times 0.05 \approx 0.050$ to $0.706 \times 0.05 \approx 0.035$—a meaningful shift that allows research tools to score relatively higher.

5.4 Results: Comparison Count Scaling

Table 17 shows how the hierarchical advantage grows with tool count.

The hierarchical advantage grows with tool count because domain pruning eliminates entire branches early. The "Reduction Factor" column indicates how many times fewer comparisons the hierarchical approach requires relative to flat retrieval (e.g., 11.9× means the hierarchical method uses ∼11.9 times fewer comparisons). At 500 tools (realistic for an MCP-connected agent with many servers), hierarchical traversal requires 12× fewer comparisons.

Table 17: Comparison count scaling: flat vs. hierarchical.

Tool Count	Flat	Hierarchical	Reduction Factor
15	15	11	1.4×
30	30	18	1.7×
50	50	22	2.3×
100	100	28	3.6×
200	200	35	5.7×
500	500	42	11.9×

5.5 Ablation Studies

We isolated the impact of each architectural component through 11 controlled benchmark runs (Table 18).

Table 18: Incremental ablation (120-query development set).

Variant	F1	Δ
Flat embedding baseline	0.345	—
+ Hierarchical traversal (single-desc.)	0.288	−0.057
+ Tight pack tool lists (remove overlap)	0.417	+0.072
+ 3-tier arch. (OS/Web/CU primitives)	0.265	−0.080
+ Exemplar-based domain matching	0.351	+0.006
+ Guaranteed OS layer inclusion	0.370	+0.025
+ Tuned domain exemplars (reduce overlap)	0.369	−0.001
+ Tiered cutoff only (no tool exemplars)	0.354	+0.009
+ Two-level exemplars + tiered cutoff	**0.448**	+0.103

Key insight. Tool-level exemplar embeddings (+0.094 from tiered cutoff baseline) are the largest single contributor to F1 improvement. The same insight that fixed domain matching (short-to-short > short-to-long) applies recursively to tools. `writeFile`'s technical description has cosine similarity ∼0.08 against business queries, while its exemplar "save this report to a file" matches at ∼0.35. No tool names are hardcoded.

Component contributions. (Table 19):

Table 19: Component contributions from 11-run analysis.

Component	Impact	Evidence
Two-level exemplar embeddings	+29.9% F1	Tool exemplars match user language
Tiered primitive cutoff	+88% Business F1	writeFile 17/20 (was 3/20)
Exemplar domain matching	+61% Research F1	Similarity ∼0.35 vs. ∼0.04
Web domain guarantee	+15% webSearch	15/20 business queries
3-tier pack separation	−30% overlap	Domains focus on unique tools
Guaranteed OS inclusion	+46% Creative F1	File tools for all domains

5.6 LLM Classification Baseline

To contextualize the "0 LLM calls" claim, we benchmarked an LLM-based tool classifier on the same 300-query dataset. The LLM receives the full tool list (19 tools) and returns a JSON array of selected tools.

Setup. Claude Opus 4 (`claude-opus-4-20250514`), temperature=0, max 200 output tokens, 300 queries—identical dataset.

Table 20: Three-way comparison: LLM vs. Skill Tree vs. flat embedding.

Metric	LLM	Skill Tree	Flat
F1	**0.661**	0.384	0.332
Precision	**0.719**	0.304	0.240
Recall	0.650	**0.660**	**0.714**
Latency	1,863 ms	**1.82 ms**	0.03 ms
LLM calls	300	**0**	0
Tokens (routing)	74,971	**0**	0
Cost (300 queries)	$0.257	**$0.00**	$0.00

Per-domain F1 comparison. (Table 21):

Table 21: Per-domain F1: LLM vs. Skill Tree (300-query).

Domain	LLM	Skill Tree	Δ
Code	**0.902**	0.456	LLM +98%
Research	**0.825**	0.422	LLM +95%
General	**0.841**	0.314	LLM +168%
Personal	**0.749**	0.476	LLM +57%
Business	**0.347**	0.316	LLM +10%
Creative	0.304	**0.323**	Skill Tree +6%

The comparison reveals five important tradeoffs.

First, the LLM achieves higher F1 (+72.1%) with substantially higher precision. The LLM selects the minimum set of tools needed; it also handles adversarial queries by returning empty tool sets. The Skill Tree is more permissive—it includes more tools (avg 4.0 per query) with reasonable coverage but more false positives. For agent systems where tool *availability* matters more than minimal selection, the higher recall may be preferable (§6.4).

Second, the gap varies by domain. On **code** and **general** queries, the LLM has a large advantage (+98% and +168% F1)—domains where natural language understanding provides a structural edge. On **business** and **creative**, the Skill Tree is competitive (gap narrows to 10%; Skill Tree exceeds LLM on creative by 6%).

Third, the token overhead is structurally significant (∼250 tokens/query for routing alone). Table 22 projects this at scale.

Fourth, the latency difference is 1,025× (1,863 ms vs. 1.82 ms). With type-time pre-computation (§3.3), the Skill Tree result is already cached by send time, reducing effective routing latency to zero.

Fifth, the two approaches differ fundamentally in adaptation capacity. The LLM's behavior is fixed by training data; improving it requires prompt engineering or fine-tuning. Skill Trees adapt

Table 22: LLM routing cost projection at scale.

Scale	Tokens/Day	Cost/Month	Skill Tree
1 user, 50 queries/day	12,500	$0.06	$0.00
1,000 users	12.5M	$62	$0.00
10,000 users	125M	$620	$0.00

continuously via Temporal EMA (+14.1% co-selection over 30 days, −29.3% for stale edges) with zero token expenditure.

In summary, Skill Trees sacrifice ∼28 percentage points of F1 (0.384 vs. 0.661) while eliminating all routing token consumption, achieving 1,025× lower latency, and providing continuous behavioral adaptation. For production systems serving thousands of users, the cumulative routing overhead of LLM classification may exceed the cost of actual task execution, making the embedding-based approach increasingly favorable at scale.

5.7 Embedding Model Ablation

To answer "is the hierarchy just compensating for a weak embedding model?", we ran the full 300-query benchmark with two models: `all-MiniLM-L6-v2` (384d, 22 MB) and `all-mpnet-base-v2` (768d, 170 MB). Both run locally via ONNX.

Table 23: Embedding model comparison (5 exemplars).

Metric	MiniLM (384-dim)	mpnet (768-dim)	Δ
Skill Tree F1	0.375	**0.458**	+22.1%
Skill Tree Precision	0.295	**0.385**	+30.5%
Skill Tree Recall	0.646	**0.721**	+11.5%
Latency	**1.67 ms**	10.1 ms	+8.4 ms
Flat F1	0.332	0.333	+0.3%
Skill Tree vs. Flat Δ	+13.0%	+37.5%	Gap **2.9×**

5.7.1 Initial Ablation: Hierarchy Amplifies Quality. The flat baseline barely changes with a better model (+0.3%), but the Skill Tree jumps +22.1%. The hierarchical exemplar matching benefits disproportionately from wider similarity distributions: better models produce more discriminative scores at each tree level, and improvements compound through two-level traversal.

Per-domain results. (Table 24):

Table 24: Per-domain F1 by embedding model (5 exemplars).

Domain	MiniLM	mpnet	Δ
Business	0.295	**0.577**	+95.6%
Creative	0.262	**0.464**	+77.1%
Research	0.468	**0.586**	+25.4%
General	0.318	0.317	−0.3%
Personal	**0.492**	0.459	−6.7%
Code	**0.417**	0.347	**−16.8%**

Business (+95.6%) and Creative (+77.1%) see the largest gains. However, Code regresses by −16.8%—an unexpected result warranting investigation.

5.7.2 Investigating the Code Regression. We analyzed the 50 code-domain queries individually. **Domain misrouting**: 11 of 24 regressed queries were routed to the *wrong domain* by mpnet (Table 25).

Table 25: Example mpnet domain misroutings (code queries).

Query	Expected	mpnet	MiniLM F1	mpnet F1
"Find where the memory leak is"	code	comp-use	75%	29%
"Add TypeScript types to handlers"	code	personal	57%	0%
"Install npm deps and update lockfile"	code	os	80%	50%
"Initialize a new git repo"	code	comp-use	33%	0%
"Extract validation into utility fn"	code	creative	29%	0%

Root cause. We analyzed the 50 code-domain queries to isolate the regression. We found that 11 of 24 regressed queries were routed to an incorrect domain by the higher-fidelity mpnet model (Table 25). The root cause lies in exemplar coverage resolution: the 5 initial code exemplars (e.g., "fix the bug…", "write a function…") adequately cover core patterns but omit DevOps, debugging, or project management intents. MiniLM's lower dimensionality (384d) functions as a semantic "blur," allowing queries broadly related to code to still match. Conversely, mpnet's higher resolution (768d) projects these edge-case queries into distinct semantic sub-regions, exposing the sparsity of the exemplar set.

5.7.3 Fix Attempt: Expanded Domain Exemplars. We expanded code exemplars from 5 to 13, adding patterns for the misrouted categories. Re-running with MiniLM confirmed exemplar expansion works (Table 26):

Table 26: Expanded exemplars: MiniLM recovery.

Metric	5 exemplars	13 exemplars	Δ
Overall F1	0.375	**0.384**	+2.4%
Code F1	0.417	**0.456**	+9.4%
Creative F1	0.262	**0.323**	+23.3%
Business F1	0.295	**0.316**	+7.1%

But for mpnet, the expanded exemplars did not fix the regression—code F1 *decreased further* (0.347 → 0.336, −3.2%), though domain misroutings improved from 11/50 to 6/50.

5.7.4 Finding: Asymmetric Model × Coverage Interaction. The complete 2 × 2 ablation reveals a non-obvious interaction (Tables 27 and 28).

Four findings emerge:

Finding 1: Hierarchy amplifies embedding quality. With expanded exemplars, the model upgrade MiniLM→mpnet improves flat retrieval by only +0.1% F1, while the Skill Tree improves by +6.6%—a **45.1× amplification ratio**. Exemplar coverage makes the hierarchy *more* important, not less.

Table 27: 2×2 ablation — Overall F1.

Config	F1	Precision	Recall	Flat	Skill Tree / Flat
MiniLM (5 exemplars)	0.375	0.295	0.646	0.332	+13.0%
MiniLM (13 exemplars)	**0.384**	**0.304**	**0.660**	0.332	+15.7%
mpnet (5 exemplars)	**0.458**	**0.385**	**0.721**	0.333	+37.5%
mpnet (13 exemplars)	0.451	0.380	0.705	0.333	+35.4%

Table 28: 2×2 ablation — Per-domain F1.

Domain	MiniLM (5 exemplars)	MiniLM (13 exemplars)	mpnet (5 exemplars)	mpnet (13 exemplars)
Code	0.417	**0.456**	0.347	0.336
Research	0.468	0.422	**0.586**	0.554
Business	0.295	0.316	**0.577**	**0.577**
Creative	0.262	0.323	**0.464**	0.460
Personal	**0.492**	0.476	0.459	0.459
General	0.318	0.314	0.317	0.319

Finding 2: Asymmetric model×coverage interaction. Expanding code exemplars from 5 to 13 improves MiniLM's code F1 by +4.0% but decreases mpnet's by −1.1%. MiniLM's lower dimensionality acts as a natural blur; mpnet's higher fidelity captures finer semantic distinctions that additional exemplars cannot bridge.

Finding 3: Model-dependent, not architectural. No single configuration maximizes both code F1 and overall F1. This suggests a natural ceiling for hand-crafted exemplars that fine-tuning can address.

Finding 4: Adversarial queries remain model-independent. General domain F1 spans 0.314–0.319 across all configurations. Zero-tool handling is an architectural gap, not a model quality gap.

5.7.5 Implications. Table 29 shows the cost-accuracy Pareto frontier.

Table 29: Cost-accuracy Pareto frontier.

Approach	F1	Latency	Cost	Size
Skill Tree + MiniLM (13 exemplars)	0.384	2 ms	$0.00	22 MB
Skill Tree + mpnet (13 exemplars)	0.451	11 ms	$0.00	170 MB
LLM (Claude Opus 4)	0.661	1,863 ms	$0.26	API

The production system uses MiniLM with expanded exemplars for its combination of best code performance and minimal footprint. A model upgrade alone—with no architectural changes—narrows the LLM gap from 28 to 21 percentage points, while remaining 170× faster and consuming zero tokens. Fine-tuning a domain-specific encoder (§6.6) is the most promising path to closing the remaining gap.

5.8 Closing the Gap: Iterative Precision Improvements

These experiments use the Phase 4 configuration (120-query development set, 5 exemplars). See §5.1 for context.

The LLM baseline reveals that the F1 gap is driven almost entirely by precision, not recall. The Skill Tree selects an average of 4.0 tools per query where the benchmark expects 2.0—a 2× over-selection ratio. We apply four progressively tighter filters.

5.8.1 Improvement 1: Query-Tool Similarity Gate. **Hypothesis**: The adaptive cutoff (μ+0.25σ) is too permissive because it is entirely relative.

Method: Introduce an absolute similarity floor τ_{base} alongside a tighter statistical cutoff:

$$\tau_{\text{tool}} = \max(\tau_{\text{base}},\ \mu_d + k \cdot \sigma_d) \tag{4}$$

After iterative tuning, we adopted $\tau_{\text{strict}} = 0.20$, $\tau_{\text{relaxed}} = 0.15$, $k_{\text{strict}} = 0.35$, $k_{\text{relaxed}} = 0.15$.

Table 30: Improvement 1: Query-tool similarity gate.

Metric	Before	After	Δ
F1	0.448	**0.463**	+3.3%
Precision	0.331	**0.351**	+6.3%
Recall	0.821	0.811	−1.2%
Avg tools/query	5.1	**4.7**	−7.8%

The result confirms that the primary bottleneck is not in the threshold mechanism but in the scoring signal itself—false positives score similarly to true positives. This motivates IDF weighting.

5.8.2 Improvement 2: Inverse Domain Frequency (IDF) Weighting. **Hypothesis**: Ubiquitous tools (`writeFile` appears in 5/7 domains) inflate selection counts.

Method: Weight tool scores by Inverse Document Frequency (IDF), analogous to TF-IDF (Term Frequency–Inverse Document Frequency) [11]:

$$s_{\text{weighted}} = s_{\text{raw}} \times \log\left(\frac{N_{\text{domains}}}{|\{d : t \in d\}|}\right) \tag{5}$$

IDF normalized to $[0.5, 1.0]$ to prevent over-penalization. After joint tuning with the statistical cutoff ($k_{\text{strict}} = 0.45$, $k_{\text{relaxed}} = 0.25$):

Table 31: Improvement 2: IDF weighting + tuned cutoff.

Metric	Before	After	Δ
F1	0.463	**0.468**	+1.1%
Precision	0.351	**0.362**	+3.1%
Recall	0.811	0.792	−2.3%
Avg tools/query	4.7	**4.4**	−6.4%

Table 32: IDF ablation: range vs. cutoff approaches.

Approach	IDF range	k	F1	Precision
IDF only	[0.5, 1.0]	0.35/0.15	0.465	0.354
Wider range	[0.3, 1.0]	0.35/0.15	0.461	0.348
Tighter cutoff	[0.5, 1.0]	**0.45/0.25**	**0.468**	**0.362**

Ablation: wider IDF range vs. tighter cutoff. Widening the IDF range hurt F1 because aggressive penalization of foundational tools removed tools that cross-domain queries genuinely require. The

tighter cutoff approach keeps IDF moderate but raises the bar for *all* tools, confirming that IDF and the statistical cutoff are complementary.

5.8.3 Improvement 3: Contrastive Negative Exemplars. **Hypothesis**: Embedding models have nonzero similarity even for unrelated pairs. "Anti-exemplars" can penalize false matches.

Method: Augment each tool node with negative exemplars and apply a contrastive penalty:

$$s_{\text{adjusted}} = s_{\text{IDF}} - \beta \cdot \max_{n \in N_t} \cos(\mathbf{e}_q, n) \tag{6}$$

We authored 2–3 negative exemplars per tool for the 11 highest over-selection offenders. We are not aware of prior work using contrastive exemplar scoring for tool routing.

Table 33: Improvement 3: Contrastive negative exemplars.

Metric	Before	After (β=0.15)	Δ
F1	0.468	**0.469**	+0.2%
Precision	0.362	**0.364**	+0.6%
Recall	0.792	0.792	0.0%
Avg tools/query	4.4	4.4	0.0

The marginal improvement exposes the *similarity banding* problem: general-purpose embedding models produce narrow similarity distributions (~0.15–0.25 even for unrelated pairs), limiting contrastive signal strength. A fine-tuned tool-routing encoder would produce wider separation, enabling higher β values.

5.8.4 Improvement 4: Complexity-Adaptive Selection Cap. **Hypothesis**: A fixed cap (MAX_TOOLS=8) is too generous for simple queries.

Method: Dynamic cap based on query complexity:

$$\text{maxTools} = f(\text{wordCount}, \text{domainConcentration}) \tag{7}$$

where domainConcentration = $(s_1 - s_2)/s_1$ measures single-domain focus.

Table 34: Complexity-adaptive selection cap thresholds.

Word Count	Concentrated	Max Tools
1–3 (short/greeting)	—	3
4–8 (medium)	Yes (> 0.15)	4
4–8 (medium)	No (< 0.15)	5
9+ (complex)	Yes	5
9+ (complex)	No	6

Table 35: Improvement 4: Complexity-adaptive cap.

Metric	Before	After	Δ
F1	0.469	**0.474**	+1.1%
Precision	0.364	**0.374**	+2.7%
Recall	0.792	0.764	−3.5%
Avg tools/query	4.4	**4.0**	−9.1%

5.8.5 Cumulative Results. Note: These results are from the 120-query development set (Phase 4, 5 exemplars). The LLM baseline F1 of 0.666 here differs from the 300-query baseline of 0.661 (§5.6) because the 120-query subset excludes some adversarial queries. See §5.1 for context.

Table 36: Cumulative precision improvements (120-query dev set).

Stage	F1	Prec.	Recall	Tools	Skill Tree vs. LLM
Baseline (§5.2)	0.448	0.331	0.821	5.1	−32.7%
+ Query-Tool Gate	0.463	0.351	0.811	4.7	−30.5%
+ IDF Weighting	0.468	0.362	0.792	4.4	−29.7%
+ Negative Exemplars	0.469	0.364	0.792	4.4	−29.6%
+ Complexity Cap	**0.474**	**0.374**	0.764	**4.0**	**−28.8%**
LLM Baseline	**0.666**	**0.790**	0.619	1.2	—

6 Discussion

6.1 Why DAGs, Not Trees

A strict tree forces artificial single-parent choices: `editFile` would have to belong to *either* Code or Business, not both. In practice, many tools are genuinely cross-domain—`deepResearch` feeds into Research AND Business AND Creative workflows. The DAG structure captures these real relationships while preventing cycles (which would create infinite traversal loops).

6.2 The Type-Time Pipeline: Latency Hiding

A notable result from a user experience perspective is that tool selection can be rendered effectively *free* at send time. By executing traversal during the keystroke debounce interval (300 ms), the Skill-Path is already cached when the user submits the message. This transforms a 15 ms operation into an effectively instantaneous one.

This is only possible because:

(1) Embedding is fast (~5 ms with a local model).
(2) Hierarchical traversal is mathematically cheap (~25 comparisons).
(3) The result is stable—partial messages converge to the same domain as the full message.

6.3 Failure Modes and Mitigations

Domain ambiguity. "Write a technical blog post about our API"—is this Code or Creative? The Skill Tree routes to Creative (primary) with Code as secondary domain, merging tools from both packs. This is correct behavior—the task is primarily creative but needs code-domain tools.

Intent modifier blindness. "Research competitors every Monday"—the Skill Tree correctly identifies Research tools but misses "every Monday" as a scheduling signal. Tier 1 (regex) catches this and overrides `executionMode` to `background`. Without Tier 1, this would execute as a one-shot research task.

Cold start. A newly installed skill pack has no co-occurrence edges. The system relies on `semantic` edges (from embedding similarity at install time) until usage data accumulates. In practice, semantic edges provide ~85% of the accuracy of mature co-occurrence edges.

6.4 Recall as the Right Metric for Routing Pre-Filters

The F1 gap between Skill Trees (0.384) and the LLM baseline (0.661) warrants careful interpretation. F1 weights precision and recall equally, but for a **routing pre-filter**—whose role is to narrow the tool inventory before the execution-stage LLM sees it—the costs of false positives and false negatives are fundamentally asymmetric.

False positives are cheap. When the Skill Tree includes an irrelevant tool (e.g., `deleteFile` for a research query), the execution-stage LLM simply ignores it. The cost is marginal context window consumption—a few hundred tokens of tool description that the LLM never invokes. In our system, tool descriptions average ∼80 tokens each; including 2 extra tools adds ∼160 tokens to an 8K+ context, a $< 2\%$ overhead that empirically does not degrade LLM task performance.

False negatives are expensive. When the Skill Tree excludes a relevant tool (e.g., omitting `webSearch` for a business analysis query), the execution-stage LLM *cannot* use it regardless of how clearly the task requires it. The cost is a failed or degraded task execution—the user notices.

Under this asymmetric cost model, **recall is the primary metric** for a routing pre-filter (Table 37):

Table 37: Recall-parity argument: asymmetric cost reframing.

Metric	Skill Tree	LLM	Gap
Recall	**0.660**	**0.650**	+1.5% (parity)
Precision	0.304	0.790	−61.5%
F1	0.384	0.661	−41.9%

Viewed through recall, Skill Trees achieve **parity** with the LLM baseline—the routing layer provides the same tool availability while being 1,025× faster and consuming zero tokens. The F1 deficit is driven entirely by precision (over-selection), whose cost is amortized by the execution-stage LLM's ability to selectively invoke only the tools it needs.

While improving precision remains an objective to further minimize context window consumption, this asymmetric cost model re-contextualizes the empirical results. In a two-stage system (pre-filter → execution LLM), the objective of the routing layer is to guarantee requisite tool *availability* with minimal latency, leaving final selection to the execution-stage LLM. Thus, while the 0.384 F1 reflects the penalty of over-selection, the recall metric confirms the routing layer successfully fulfills its architectural role.

6.5 Limitations

(1) **Evaluation scale**: Our benchmark covers 300 queries across 6 domains with 18 tools. Larger-scale evaluation with hundreds of tools and thousands of queries would further strengthen the claims. Existing benchmarks (ToolBench: 16K APIs; MetaTool: 21K queries; ToolRet [10]: 43K tools) evaluate execution-time tool invocation or retrieval model quality rather than routing-layer pre-filtering.

(2) **User study**: The 30-day adaptation results use simulated usage. We have designed a two-phase validation study: Phase 1 (structured, N=8–12 developers, 20 tasks across 6 domains) measures real-world tool selection F1, and Phase 2 (longitudinal, 30 days of natural usage) tracks co-selection accuracy improvement over weekly checkpoints. The study is planned for post-launch recruitment from the open-source community.

(3) **Adversarial query handling**: The system currently has no mechanism to produce an empty tool set. Zero-tool queries (greetings, one-word responses) account for 23 of 300 benchmark queries and always receive 3+ spurious selections, regardless of embedding model quality (§5.7). This is an architectural gap—no embedding model can produce "select nothing" behavior without an explicit confidence gate.

(4) **Embedding model trade-off**: We use `all-MiniLM-L6-v2` (384d) in production for its minimal footprint (22 MB, 1.7 ms). The ablation (§5.7) shows that upgrading to `all-mpnet-base-v2` (768d) improves overall F1 by 17.4% at the cost of +8.4 ms latency and 170 MB, but introduces a code domain regression (−19.4%). Domain-specific fine-tuning represents the highest-impact future work.

(5) **Code domain regression**: The 2×2 ablation (§5.7) reveals a resolution-coverage interaction: mpnet's Code F1 (0.336–0.347) is consistently lower than MiniLM's (0.417–0.456), despite mpnet's superior overall performance. This demonstrates that higher-dimensional models capture finer semantic distinctions that exceed the descriptive capacity of hand-crafted exemplars. The fix path is a fine-tuned tool-routing encoder, not additional exemplar engineering.

6.6 Generalizability and Future Work

The Skill Tree architecture is not specific to NIOM. The core requirements are minimal: a set of tools with textual descriptions, a lightweight embedding model, and a DAG structure mapping domains to tools.

MCP-connected agents. The Model Context Protocol [13] allows agents to dynamically connect to tool servers, potentially accumulating hundreds of tools. NIOM already indexes MCP-discovered tools into the Skill Tree at connection time. For other MCP clients, the adoption path is straightforward: embed each tool's `description` field as a tool-level exemplar, group tools by server as a proxy for domain, and run the same two-level traversal. The scaling analysis (§5.4) shows that hierarchical traversal's advantage grows with tool count—at 500 tools, 12× fewer comparisons than flat retrieval.

Enterprise agent platforms. Platforms like ServiceNow and Salesforce ship domain-specific toolkits (IT operations, customer service, sales automation). These naturally map to the 3-tier pack architecture: platform-level primitives as the base layer, domain toolkits as intent packs, and a general fallback. The tiered cutoff mechanism would require adaptation—enterprise domains may have different overlap patterns—but the architectural pattern transfers.

Customization without retraining. A structural advantage of Skill Trees is that routing behavior can be customized entirely through configuration—editing the DAG structure, adding exemplar queries, toggling domains, or adjusting edge weights—without any model

retraining, prompt engineering, or additional token expenditure. In LLM-based classification, customizing tool selection requires modifying prompts (consuming tokens per query), adding few-shot examples (ongoing cost), or fine-tuning (requiring ML expertise and GPU resources). In trajectory-based approaches like AutoTool [1] or SkillRL [12], customization requires generating new behavioral traces and retraining. Skill Trees reduce this to a configuration-level operation: an enterprise can reparent a tool node, add 3–5 exemplar queries to change matching behavior, or disable entire capability branches—changes that take effect at the next re-index with zero cost. This makes routing customization accessible to non-ML practitioners and supports scenarios where tool inventories change frequently, different teams require different routing profiles, or compliance constraints require specific exclusions.

Multi-agent orchestration. The DAG structure extends beyond tool routing to agent routing: in a multi-agent system, each specialized agent can be treated as a "tool" with its own exemplar embeddings. The hierarchical traversal would first identify the relevant agent domain, then select the specific agent—the same two-level narrowing applied one level up.

What would need to change. The exemplar strategy assumes a moderate tool inventory (18–200). At very large scale (1,000+ tools with fine-grained overlap), the two-level hierarchy may need a third level (domain → subdomain → tool) or the similarity banding problem (§5.8) may require fine-tuned embeddings. The 3-tier cutoff thresholds were tuned for NIOM's domain distribution and would need re-calibration for different systems.

7 Conclusion

We have presented Skill Trees, a hierarchical DAG-based tool selection mechanism for LLM agents that addresses the tool scaling bottleneck through three mechanisms: (1) hierarchical traversal that narrows 200+ tools to 3–8 candidates via logarithmic embedding comparisons, (2) type-time pre-computation that entirely masks routing latency for over 80% of requests, and (3) self-improving co-occurrence edges that increase tool co-selection accuracy by 14.1% over 30 simulated days.

The three-phase pipeline—install-time indexing, type-time traversal, and send-time injection—provides a deployable alternative to both flat embedding retrieval and LLM-based classification. On a 300-query benchmark featuring adversarial, cross-domain, and zero-tool queries, Skill Trees achieve recall parity with the LLM baseline (0.660 vs. 0.650) while reducing routing latency by three orders of magnitude (1,025×) and consuming zero routing tokens. Under the asymmetric cost model of a two-stage routing architecture, this trade-off heavily favors hierarchical embedding pre-filters at scale.

Table 38: Final comparison across all evaluated approaches.

Approach	F1	Latency	Cost	LLM
Skill Tree + MiniLM (13 exemplars)	0.384	2 ms	$0	0
Skill Tree + mpnet (13 exemplars)	0.451	11 ms	$0	0
Flat Embedding	0.332	<1 ms	$0	0
LLM (Claude Opus 4)	0.661	1,863 ms	$0.26	300

An empirical analysis of the 2×2 embedding model ablation reveals that the hierarchical architecture **amplifies** embedding quality with a 45.1× ratio over flat retrieval. We further observe an asymmetric interaction between model dimensionality and exemplar coverage, indicating that fine-tuned routing encoders, rather than manual exemplar expansion, represent the optimal path to closing the remaining precision gap.

The system's capacity to adapt automatically to evolving user behavior via temporal EMA edge learning further differentiates it from static LLM routing constraints. These properties—zero-cost routing, sub-2 ms latency, and continuous adaptation—become critical at scale, where the cumulative routing token overhead of LLM classification may exceed the cost of actual task execution.

Future Work:

(1) Hierarchical traversal with larger tool inventories (500+).
(2) A fine-tuned tool-routing encoder using contrastive learning (Multiple Negatives Ranking Loss) on benchmark-derived triplets with LLM-augmented paraphrases (∼14,000 training pairs), targeting 3.5× wider similarity separation that would enable stronger contrastive penalties ($\beta = 0.3$–0.5) and more discriminative thresholds—conservatively estimated to narrow the LLM F1 gap from 42% to 15–25%.
(3) An explicit confidence gate for zero-tool queries (greetings, conversational input).
(4) A two-phase user study (N=8–12 developers: 1-hour structured session + 30-day longitudinal usage) measuring real-world tool selection F1, co-selection improvement over time, and type-time SkillPath stability.
(5) Cross-device Skill Tree synchronization.
(6) Integration with AutoTool-style [1] sequential prediction for multi-step trajectory refinement.

References

[1] Zhao, M. et al. (2025). AutoTool: Efficient Tool Selection for Large Language Model Agents via Automatic Tool Retrieval. *Proceedings of AAAI 2026*. arXiv:2511.14650.
[2] Qin, Y. et al. (2024). ToolLLM: Facilitating Large Language Models to Master 16000+ Real-World APIs. *Proceedings of ICLR 2024*. arXiv:2307.16789.
[3] Huang, Y. et al. (2024). MetaTool Benchmark for Large Language Models: Deciding Whether to Use Tools and Which to Use. *ACL 2024 Findings*. arXiv:2310.03128.
[4] Apple (2024). ToolSandbox: A Stateful, Conversational, Interactive Evaluation Framework for LLM Tool Use Capabilities. *arXiv preprint*, arXiv:2408.04682.
[5] Berkeley Function Calling Leaderboard (2024). gorilla-llm.github.io/leaderboard.
[6] Zhang, Y. et al. (2025). Agent-as-a-Graph: Agent Architecture with Retrieval and Hierarchical Graph Structures. *arXiv preprint*, arXiv:2502.11403.
[7] Zhang, Z. et al. (2025). AgentRouter: A Knowledge-Graph-Guided LLM Router for Collaborative Multi-Agent Question Answering. *arXiv preprint*.
[8] Chen, J. et al. (2026). A-RAG: Scaling Agentic Retrieval-Augmented Generation via Hierarchical Retrieval Interfaces. *arXiv preprint*, arXiv:2502.11356.
[9] TURA (2025). Tool Use Reliability Assessment for LLM Agents. *arXiv preprint*.
[10] Shi, Z., Wang, Y., Yan, L., Ren, P., Wang, S., Yin, D., and Ren, Z. (2025). Retrieval Models Aren't Tool-Savvy: Benchmarking Tool Retrieval for Large Language Models. *arXiv preprint*, arXiv:2503.01763.
[11] Moon, S. et al. (2024). Efficient and Scalable Estimation of Tool Representations in Vector Space. *arXiv preprint*, arXiv:2409.02141.
[12] Xia, P. et al. (2026). SkillRL: Evolving Agents via Recursive Skill-Augmented Reinforcement Learning. *arXiv preprint*, arXiv:2602.08234.
[13] Anthropic (2024). Model Context Protocol Specification. modelcontextprotocol.io.
[14] Krishnan, N. (2025). Advancing Multi-Agent Systems Through Model Context Protocol. *arXiv preprint*, arXiv:2504.21030.
[15] Reimers, N. and Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *Proceedings of EMNLP 2019*. arXiv:1908.10084.
[16] Vercel (2025). AI SDK v6 Documentation. sdk.vercel.ai.

[17] LangChain (2024). LangChain: Building applications with LLMs through composability. github.com/langchain-ai/langchain.
[18] Tennenhouse, D. (2000). Proactive Computing. *Communications of the ACM*, 43(5), 43–50.
[19] Weiser, M. (1991). The Computer for the 21st Century. *Scientific American*, 265(3), 94–104.
[20] Huang, J. et al. (2026). Graph-Based Self-Healing Tool Routing for LLM Agents. *arXiv preprint*.
[21] Li, S. et al. (2025). GraphRouter: A Graph-based Approach for Adaptive LLM Routing. *arXiv preprint*.
[22] Chen, Y. et al. (2026). HAPS: Hierarchical LLM Routing with Joint Architecture and Parameter Search. *arXiv preprint*.